1. Technical Objectives

1-1. How do you audit control effectiveness against an actual incident timeline?

A control effectiveness audit checks whether security controls actually worked during an incident.

A control might be installed and enabled but still fail to detect or stop an attack.

To audit controls:

1. Review the incident timeline to understand what the attacker did and when.
2. Identify which security controls should have detected or blocked each action.
3. Check logs and configurations to see whether those controls worked.
4. Identify failures and their causes.
5. Document improvements and how to test them.

Example:

An attacker executes a malicious file using MSBuild, a legitimate Windows development tool.

Expected control	Actual result	Finding
Sysmon logs process execution	Event recorded	Working
SIEM detects suspicious MSBuild activity	No alert	Detection gap
Endpoint protection blocks malicious execution	Execution allowed	Prevention gap

The audit shows that logging worked, but detection and prevention need improvement.

Important: A missing alert does not always mean the detection rule failed. The necessary logs might never have reached the SIEM.

1-2. How do you write Sigma detection rules targeting observed TTPs?

Sigma is an open detection-rule format written in YAML. It helps security teams describe suspicious activity in logs.

A Sigma rule can be converted into queries for supported SIEM platforms.

TTPs are:

- Tactics: What the attacker wants to achieve.

- Techniques: How the attacker achieves it.
- Procedures: The specific actions and tools used.

For example, MITRE ATT&CK technique T1127.001 (MSBuild) describes how attackers may abuse a trusted development utility to execute malicious code.

A simplified Sigma rule:

```
title: Suspicious MSBuild Execution
id: 8f28b2c1-6a14-4e7d-91a2-52e43c9d8f10
status: experimental
description: Detects MSBuild launched from a user temporary directory
logsource:
  product: windows
  category: process_creation
detection:
  selection:
    Image|endswith: '\MSBuild.exe'
    CurrentDirectory|contains: '\AppData\Local\Temp\'
  condition: selection
level: medium
tags:
  - attack.t1127.001
```

This example looks for MSBuild running from a temporary working directory, which may be suspicious.

However, MSBuild is a legitimate tool. Its execution alone does not prove malicious activity.

The rule should be tested against real incident events and normal developer activity before deployment.

1-3. How do you test detection rules using positive and negative samples?

Detection testing checks whether a rule detects attacks without generating too many unnecessary alerts.

Two datasets are used:

- Positive samples: Logs containing known malicious activity, such as `incident_sysmon.jsonl`.
- Negative samples: Logs containing normal activity, such as `clean_sysmon.jsonl`.

Four possible outcomes exist:

Outcome	Meaning
True Positive (TP)	Attack happened, and the rule detected it
False Positive (FP)	Normal activity triggered an alert
True Negative (TN)	Normal activity correctly produced no alert
False Negative (FN)	Attack happened, but the rule missed it

False Positive Rate (FPR):

[FPR=\frac{FP}{FP+TN}\times100]

Example: A rule is tested against 1,000 normal events. It incorrectly alerts on 20.

[FPR=\frac{20}{1000}\times100=2%]

The false positive rate is 2%.

You should also measure how many real attacks the rule detects.

Detection Rate (Recall):

[Recall=\frac{TP}{TP+FN}\times100]

For example, detecting 18 out of 20 known malicious events gives a 90% detection rate.

A good rule should detect as many relevant attacks as possible while keeping false alerts manageable.

Note: These calculations assume the tested events have been correctly labeled as malicious or benign. A clean baseline alone does not guarantee every event is harmless.

1-4. How do you enrich IOCs and TTPs with open-source threat intelligence?

Threat intelligence enrichment means adding useful external information to suspicious indicators and attacker techniques.

An IOC (Indicator of Compromise) is evidence that may indicate malicious activity.

Common IOCs include IP addresses, domains, file hashes, and suspicious URLs.

Useful intelligence sources:

Source	What it provides
VirusTotal	File, URL, domain, and IP reputation information
AlienVault OTX	Community threat indicators and threat reports

Source	What it provides
AbuseIPDB	Reports of potentially abusive IP addresses
MITRE ATT&CK	Attacker techniques, behaviors, and known group associations

Example:

During an investigation, you find a suspicious IP address communicating with an infected computer.

You can:

1. Search the IP in AbuseIPDB and OTX.
2. Check whether other researchers have reported malicious activity.
3. Review any related malware or attack techniques.
4. Compare those findings with your incident evidence.
5. Identify additional indicators worth monitoring.

Important: A malicious reputation score does not prove an IP participated in your incident. Likewise, matching a known group's techniques does not prove that group was responsible.

1-5. How do you produce hardening specifications based on incident findings?

Hardening means reducing weaknesses and unnecessary attack opportunities in a system.

A hardening specification describes exactly which security settings must change.

It should connect every change to a specific incident finding.

Example:

An attacker used an ordinary employee account to run an unauthorized program.

Incident finding	Hardening improvement
Users can execute unapproved applications	Introduce application control
Dangerous child processes are allowed	Evaluate relevant Microsoft ASR rules
Excessive administrator privileges	Apply least privilege
Security logging is incomplete	Enable required auditing

A good specification explains:

- The affected system and setting.
- The required configuration change.

- The reason for the change.
- The engineer responsible.
- How to test that the change works.

For Windows environments, Microsoft Attack Surface Reduction (ASR) rules and CIS Benchmarks provide useful hardening references.

For example, an ASR rule may help block certain suspicious Office application behaviors. The engineer should first check compatibility, test the rule in audit mode where supported, and then consider enforcement.

2. Conceptual Objectives

2-1. What is the difference between controls in place and controls that work?

A control in place is a security measure that has been installed or configured.

A working control is one that actually performs its intended security function.

For example:

- A firewall is installed, but its rules allow unnecessary traffic.
- An antivirus product is active, but it fails to stop malicious execution.
- A SIEM collects logs, but no rule detects the attack.

All three controls exist, but their effectiveness is limited.

Security teams should therefore measure actual results, not just whether a product or setting is present.

This is why post-incident audits are important: they reveal the difference between assumed protection and proven protection.

2-2. What is the detection engineering lifecycle?

Detection engineering is the process of developing, testing, deploying, and maintaining rules that identify suspicious activity.

It follows four main stages.

1

Write

Create rule

2

Test

Check results

3

Tune

Reduce errors

4

Deploy

Use in SOC

Monitor results and repeat when needed

1. Write: Create a detection rule based on observed attacker behavior.
2. Test: Run it against malicious and clean samples.
3. Tune: Adjust the conditions to reduce false positives without missing real attacks.
4. Deploy: Activate the validated rule in the production SIEM and monitor its performance.

Example:

A rule detects every execution of PowerShell, generating hundreds of harmless alerts.

Instead of removing the rule, the engineer tunes it to look for more suspicious behavior, such as encoded commands combined with unusual process relationships.

After testing, the improved rule is deployed.

Detection engineering is continuous because attacker behavior, software, and normal business activity change.

2-3. Why is threat intelligence forward-looking rather than only retrospective?

Retrospective analysis investigates what already happened.

Forward-looking threat intelligence uses available information to prepare for what an attacker might do next.

For example, an investigation identifies the use of stolen Kerberos tickets.

MITRE ATT&CK describes this behavior under T1550.003 — Pass-the-Ticket.

Instead of only documenting the technique, defenders can:

- Investigate related authentication activity.

- Review other systems for similar behavior.
- Improve monitoring of suspicious ticket use.
- Strengthen credential and account protections.

Threat intelligence helps teams move from reacting to attacks toward anticipating plausible future threats.

However, these are possible next steps, not predictions that an attacker will definitely follow.

2-4. Why should a risk register be a living document?

A risk register records identified security risks and how the organization plans to manage them.

Typical information includes:

- Risk description and affected assets.
- Likelihood and potential impact.
- Risk level and existing controls.
- Risk owner and treatment plan.
- Deadline and current status.

It is called a living document because it must change when new evidence becomes available.

Example:

Before an incident, an organization considers credential theft a medium risk.

After an investigation confirms that stolen credentials allowed access to critical systems, the team reassesses the likelihood, impact, and effectiveness of existing controls.

The risk may now require higher priority and additional protection.

The important point is that real incidents provide evidence for updating risk assessments, rather than relying only on earlier assumptions.

3. Transversal Objectives

3-1. Why must every improvement have an owner, deadline, and validation method?

A recommendation is not useful unless someone is responsible for implementing it.

Every improvement needs three things:

Element	Purpose	Example
Owner	Who implements it?	SOC Engineer
Deadline	When must it be completed?	Within 7 days
Validation	How do we confirm it works?	Replay known malicious events

For example:

Improvement: Deploy a Sigma rule detecting suspicious MSBuild execution.

Owner: SOC Detection Engineer.

Deadline: Seven days after approval.

Validation: Confirm that the rule detects the known malicious sample and produces an acceptable false positive rate on clean data.

Without these details, improvements can remain unfinished or unverified.

3-2. What does it mean to bound conclusions to evidence and recommendations to findings?

It means only claiming what the evidence supports and recommending changes that address confirmed problems.

For example:

Evidence: Sysmon recorded suspicious MSBuild execution, but no SIEM alert was generated.

Supported conclusion: The activity was not successfully alerted on in the SIEM.

Appropriate recommendation: Investigate the logging and detection pipeline, then create or repair the relevant rule.

Unsupported conclusion: The SIEM was completely broken.

That final statement is too broad because one missing alert does not prove that every SIEM function failed.

Good post-incident reports clearly distinguish between confirmed facts, reasonable interpretations, and unresolved questions.

3-3. How do you write artifacts other engineers can execute without asking questions?

An artifact is a practical deliverable, such as a detection rule, configuration file, hardening specification, test report, or incident response playbook.

A useful artifact should contain enough detail for another engineer to understand, implement, and verify it.

For example, a detection-rule package should explain:

1. Purpose: What behavior does the rule detect?
2. Requirements: Which logs, fields, and tools are needed?
3. Implementation: Where and how should the rule be deployed?
4. Testing: Which positive and negative samples should be used?
5. Expected results: What should trigger an alert?
6. Rollback: How can the change be safely reversed if necessary?

Consider the difference:

Poor instruction: Improve Windows logging.

Executable instruction: Enable the required Sysmon process-creation logging on the affected Windows endpoints, confirm the events reach the SIEM, and test using a known process-creation event.

The second instruction is measurable and can be implemented without guessing what was intended.

4. Applying Everything to the Project

The purpose of this project is to convert forensic findings into security improvements that can actually be deployed.

The following example shows how the learning objectives connect.

4-1. Example: Improving defenses after an MSBuild attack

Imagine the forensic investigation confirms that an attacker abused MSBuild to execute malicious code, but the SOC received no alert.

Stage	Action	Deliverable
1. Control audit	Investigate why the activity was not detected	Control gap report
2. Detection development	Create a Sigma rule for suspicious MSBuild behavior	Sigma YAML file
3. Detection testing	Test against incident and clean Sysmon logs	Test results and FPR
4. Threat intelligence	Research related indicators and techniques	IOC enrichment report

Stage	Action	Deliverable
5. Hardening	Restrict unnecessary execution paths	Hardening specification
6. Risk management	Reassess the risk of trusted-tool abuse	Updated risk register
7. Incident response	Update investigation and escalation instructions	Revised IR playbook
8. Verification	Confirm that changes are implemented and effective	Validation evidence

The final result should not be a report saying, "We need better security."

It should be a working detection rule, tested configurations, updated procedures, documented responsibilities, and evidence that the improvements are effective.

5. Key Terms Reference

Term	Simple definition
PIR	Post-Incident Review: examining an incident to identify improvements
Security Control	A measure that prevents, detects, or reduces security risks
Control Gap	A weakness or missing capability in a security control
Sigma	A standardized format for writing log detection rules
TTP	Tactics, Techniques, and Procedures used by attackers
IOC	Indicator of Compromise, such as a suspicious IP or file hash
Threat Intelligence	Information used to understand and prepare for cyber threats
False Positive	An alert triggered by non-malicious activity
False Negative	Malicious activity that a detection fails to identify
FPR	False Positive Rate: the proportion of benign samples incorrectly alerted on
Hardening	Strengthening a system by reducing security weaknesses
ASR	Attack Surface Reduction: protections that limit risky behaviors
Risk Register	A document used to track and manage identified risks
IR Playbook	Step-by-step instructions for handling a particular incident type
Validation	Testing whether a security improvement works as intended

Final takeaway

The most important lesson of this project is that an incident should lead to measurable security improvements.

A successful post-incident process connects:

Evidence → Findings → Improvements → Implementation → Testing → Verification

The organization does not become more secure simply because it understands the attack. It becomes more secure when it uses that understanding to improve its controls and confirms that those improvements work.